## Initial Decision Matrix: Technical Architecture

This document tracks the core technical choices made at the start of the project, including the tradeoffs considered and the final path selected.

| Feature | Option A | Option B | Selected Path | Engineering Reasoning |
|---|---|---|---|---|
| **Orchestration** | LangChain / Frameworks | Scratch-built SDK implementation | **Scratch (Gemini SDK)** | Shows deeper understanding of the message loop and state management. Better for a "Quant Engineer" role to demonstrate logic control. |
| **SQL Access** | Text-to-SQL (Open) | Pre-defined Functions (Rigid) | **Validated Text-to-SQL** | A hybrid "middle ground." Allows flexibility for "unseen queries" but uses a Python-based validator to block destructive commands and enforce limits. |
| **Vector DB** | Managed Cloud (Pinecone) | Local-first (ChromaDB) | **ChromaDB** | Industry standard for local/containerized RAG. Avoids 3rd party cloud dependencies while demonstrating high-quality retrieval architecture. |
| **Access Model** | Full Context Exposure | Tiered Access Masking | **Tiered Masking** | The LLM is treated as a "masked" entity with tool-only access. The User and Backend have full visibility, ensuring the model can't leak the entire dataset. |
| **Database** | PostgreSQL | SQLite | **SQLite** | Priority on "Deployment Readiness" and ease of evaluation. One-file DB means the interviewer can run the app instantly without DB config friction. |

## Post-Decision Requirements

1. **Trace View:** Must map LLM outputs back to specific PDF page/snippet IDs without exposing the whole doc to the model in the prompt.

2. **Double-Request Guard:** Implement debouncing/loading states on the UI to prevent "chaotic" user input issues.
3. **Data Consistency:** Mock data generation script must ensure the CSVs and PDFs tell the same story (e.g., matching titles and trends).